THOMAS MORE

Management Rapportage Tool

Realisatiedocument

Tom Gijsbers
Student Bachelor in de Toegepaste Informatica – Applicatieontwikkeling

2025 - 2026

Inhoudsopgave

1. INLEIDING 3

2. GEBRUIKTE TECHNOLOGIEËN EN ARCHITECTURALE KEUZES 4

2.1. Inleiding 4

2.2. Backend architectuur: Van fat controller naar service objects 4

2.3. Datatransport: Gestructureerde overdracht 4

2.4. Frontend technologie: stimulus & fetch API vs. turbo frames 5

2.5. Export strategie: Loskoppelen van weergave en logica 6

2.6. View-architectuur: Het scherm opknippen in componenten 6

3. FUNCTIONEEL ONTWERP 7

3.1. Tijdsperiodeselectie 7

3.2. Dynamische filters & status 8

3.3. Tabelinteractie & sortering 9

3.4. Layout & scrolling 9

3.5. Data-integriteit & waarschuwingen 10

3.6. Export naar excel 10

4. TECHNISCHE REALISATIE 11

4.1. Backend architectuur: Overzichtelijke en modulaire code 11

4.1.1. Scheiding van Verantwoordelijkheden 12

4.2. Snelheidsoptimalisatie en database-efficiëntie 13

4.2.1. Definitieve oplossingen 14

4.3. Frontend realisatie 15

4.3.1. View refactoring 15

4.3.2. Frontend logica: De rapportage-controller 16

4.3.3. UX-optimalisatie 16

4.4. Stabiliteit 17

4.5. Slimme Excel-export op basis van schermweergave 18

4.6. Toegangsbeveiliging en Gebruikersrechten 18

5. KWALITEITSBORGING EN REFACTORING 19

5.1. Code review proces & peer feedback 19

5.2. UI/UX Feedback en Iteratie 20

5.3. Geautomatiseerde unit-test 21

5.4. Controle datakwaliteit 21

6. CONCLUSIE EN REFLECTIE 22

LITERATUURLIJST 23

BIJLAGE A — BESTANDSSTRUCTUUR MANAGEMENT RAPPORTAGE MODULE. 24

1. Inleiding

Voorafgaand aan de uitwerking van deze stageopdracht is een plan van aanpak opgesteld, waarin de scope en doelstellingen helder zijn vastgelegd. Dit plan vormde de blauwdruk voor de uiteindelijke opdracht: de ontwikkeling van een geautomatiseerde management rapportage tool binnen de *fouriq_portals_engine*. Het doel van deze module is om complexe verbruiks-, klimaat- en WEii-data van meerdere gebouwen samen te voegen tot één overkoepelend dashboard. Hierdoor kunnen vastgoedbeheerders autonoom rapportages genereren en exporteren naar Excel. De module fungeert als een 'Orchestrator' en vervangt een voorheen tijdrovend en foutgevoelig handmatig rapportageproces.

Dit document beschrijft de technische realisatie van dit nieuwe rapportagedashboard, inclusief de gemaakte architectuurkeuzes en de toegepaste defensieve programmeertechnieken.

In hoofdstuk 2 worden de gemaakte architecturale en technologische ontwerpkeuzes toegelicht en verantwoord. In hoofdstuk 3 wordt dieper ingegaan op het functioneel ontwerp. In hoofdstuk 4 wordt de daadwerkelijke implementatie van de code en de frontend-weergave besproken. In hoofdstuk 5 worden het testproces, de code reviews en de datakwaliteit besproken. Tot slot volgt in hoofdstuk 6 een conclusie en een persoonlijke reflectie op het project en de leerdoelen.

AI-Gedreven Ontwikkeling: Dit realisatiedocument en de bijbehorende processen zijn deels tot stand gekomen en geoptimaliseerd met ondersteuning van geavanceerde AI-tools.

2. Gebruikte technologieën en architecturale keuzes

2.1. Inleiding

In dit hoofdstuk worden de analyses en architecturale keuzes beschreven die ter voorbereiding op de effectieve implementatie zijn gemaakt. Voor de ontwikkeling van een schaalbare rapportagetool was het noodzakelijk om bestaande patronen binnen de applicatie kritisch te evalueren. Hieronder worden de keuzes omtrent de backend-architectuur, het datatransport en de frontend-technologieën toegelicht.

2.2. Backend architectuur: Van fat controller naar service objects

De applicatie is gebouwd bovenop het bestaande Ruby on Rails (MVC) framework. Echter, wegens de complexiteit van de datastroom zoals het ophalen van data uit Redis, het berekenen van WEII-standaarden en het aggregeren van sensordata ontstond het risico op een zogenaamde *"Fat Controller".*
Wanneer alle bedrijfslogica in de controller of het model wordt geplaatst, overtreedt dit het *Single Responsibility Principle*, wat de code onleesbaar en moeilijk testbaar maakt. Om dit te voorkomen is er gekozen voor de implementatie van het Service Object Pattern (Norman, 2023**)**. De rekenlogica en data-extractie zijn hierdoor volledig geïsoleerd in specifieke diensten (zoals de ManagementReportService). De controller is nu enkel nog verantwoordelijk voor het afhandelen van het HTTP-verzoek en het aanroepen van de juiste service.

2.3. Datatransport: Gestructureerde overdracht

Om de communicatie tussen de backend (services) en de frontend (Views en de Excel-export) robuust en voorspelbaar te maken, is een ‘Plain Old Ruby Object’ (PORO) ontworpen: ManagementReport. Dit object fungeert als een Data Transfer Object (DTO). Het ontvangt de ruwe, berekende data vanuit de services en transporteert dit gestructureerd naar de view. Omdat deze data puur dient voor rapportagedoeleinden en niet permanent opgeslagen hoeft te worden, was het aanmaken van een nieuwe databasetabel (ActiveRecord) onnodig en inefficiënt. Dit sluit aan bij het YAGNI-principe (You Aren't Gonna Need It uit de pragmatische programmeerfilosofie van Thomas & Hunt (1999).

2.4. Frontend technologie: stimulus & fetch API vs. turbo frames

Voor de weergave van de data in de browser moest een keuze worden gemaakt voor de laadstrategie. Omdat het dashboard data van tientallen tot honderden gebouwen moet aggregeren, zou het synchroon inladen van de pagina resulteren in onacceptabel lange wachttijden voor de gebruiker. Om de *performance* hoog te houden, zijn drie verschillende strategieën geëvalueerd:

1. Klassieke synchrone rendering
In de eerste iteratie werd geprobeerd alle data direct vanuit de controller aan de view mee te geven.

- Voordeel*:* Zeer simpele implementatie.
- Nadeel*:* De server moest alle externe services en database-query's afwachten voordat de pagina überhaupt naar de browser werd gestuurd. Dit leidde tot time-outs bij grotere datasets.

2. Turbo Frames
Binnen het moderne Rails Hotwire-ecosysteem is Turbo Frames de standaard manier om delen van een pagina asynchroon in te laden (lazy loading) zonder JavaScript te schrijven (Hotwire, z.d.).

- Voordeel*:* Geen custom JavaScript nodig; naadloze integratie met Rails.
- Nadeel*:* Turbo Frames missen fijnmazige controle. Bij 100 gebouwen vuurde Turbo direct 100 verzoeken tegelijk af op de server (een *request storm*). Bovendien was client-side sorteren (kolommen ordenen zonder de server opnieuw aan te spreken) extreem lastig te realiseren met puur Turbo.

3. StimulusJS gecombineerd met de Fetch API.
In de uiteindelijke iteratie is er gekozen voor een custom Stimulus-controller in combinatie met asynchrone fetch requests.

- Voordeel*:* maximale controle. De Stimulus-controller detecteert wanneer een gebouw-rij in beeld komt en vuurt pas op dat moment een gerichte fetch-request af naar de server (loadBuildingRow). Daarnaast houdt Stimulus de staat van de tabel in het browsergeheugen bij, waardoor functionaliteiten zoals client-side sorteren (reSort) en het dynamisch genereren van een export-formulier ('Smart Export') mogelijk werden zonder de server te belasten.

Besluit op basis van tabel 1: Hoewel Turbo Frames aantrekkelijk waren vanwege de snelle implementatie, blijkt uit tabel 1 dat Stimulus in combinatie met de Fetch API de enige haalbare keuze is. Het biedt de benodigde controle om rijen asynchroon (één voor één via XHR) op te halen en maakt complexe client-side interacties, zoals sorteren en slimme export-functionaliteiten, mogelijk.

Criteria	Weight	Sync. Rendering	Turbo Frames	Stimulus + Fetch API
Integratiegemak	5	9 (45)	8 (40)	7 (35)
Controle over asynchroon laden	7	3 (21)	4 (28)	9 (63)
Client-side sorteren	3	1 (3)	3 (9)	7 (21)
Custom export logica	4	3 (12)	5 (20)	8 (32)
Total Score	**19**	**81** **43%**	**97** **51%**	**151** **79%**

Recommended Decision:

Stimulus + Fetch API (Score: 79%)

Tabel 1: Decision matrix frontend technologie.

2.5. Export strategie: Loskoppelen van weergave en logica

In de initiële iteratie werd de gem caxlsx_rails overwogen om de Excel-rapportage direct vanuit de Rails-views te genereren (via zogenaamde .axlsx templates). Hoewel dit in het begin snel resultaat leverde, ontstond er een architecturaal probleem: de complexe logica voor het opmaken van de Excel-sheet raakte te sterk gekoppeld aan de view-laag. Dit is een overtreding van het *Single Responsibility Principle*.

Naarmate de architectuur verschoof naar service objects, is er besloten om af te stappen van deze view-gebaseerde aanpak. In plaats daarvan is een losgekoppelde '*ManagementReportExportService*' ontwikkeld. Binnen deze service wordt nog steeds gebruikgemaakt van de onderliggende caxlsx-bibliotheek, maar dan geïmplementeerd als pure Ruby-code op de achtergrond.

Deze service maakt direct gebruik van de gegenereerde DTO's (*ManagementReport*) om programmatisch een Excel-sheet op te bouwen die exact overeenkomt met de datastructuur op het scherm. Dit resulteert in een volledige ontkoppeling ("decoupling") van frontend en backend, wat leidt tot een schonere, beter testbare en onderhoudbare codebase.

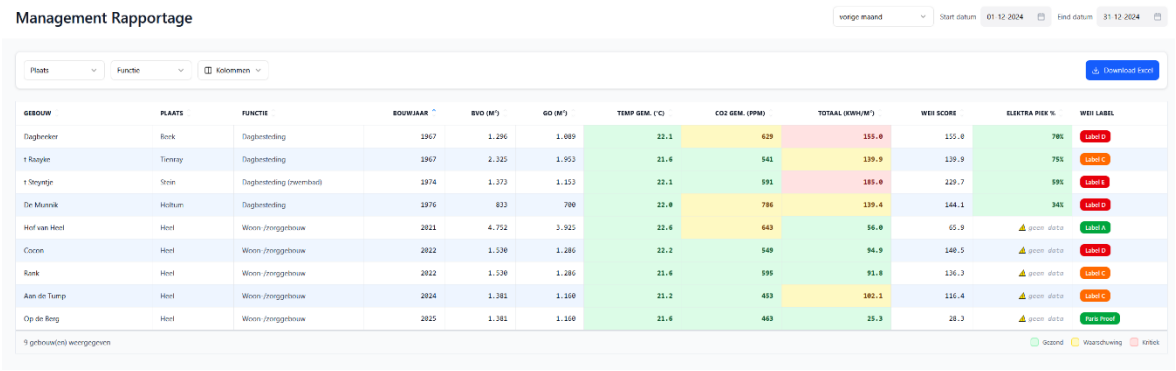
2.6. View-architectuur: Het scherm opknippen in componenten

Naast de keuze voor de JavaScript-technologie, moest ook de architectuur van de HTML-weergave (de Views) worden geanalyseerd. Binnen een traditionele Rails-applicatie bestaat het risico dat een complex dashboard resulteert in één massief, onleesbaar HTML-bestand (spaghetti-code).

Om de codebase onderhoudbaar te houden en te voldoen aan het DRY-principe, is besloten om de frontend op te knippen in kleine, modulaire componenten (partials). Deze componentisatie dient een cruciaal tweede doel: het faciliteert de asynchrone 'lazy loading' strategie. Om de Stimulus-controller in staat te stellen dynamisch één specifieke gebouw-rij in te laden via een netwerkverzoek, moet de server in staat zijn om uitsluitend de HTML van die specifieke rij te renderen en te retourneren, zonder de omliggende lay-out of headers mee te sturen. Een strikte scheiding in partials is hiervoor een absolute technische vereiste.

3. Functioneel ontwerp

Om de functionele eisen visueel te vertalen, is een interface-ontwerp opgesteld. Dit ontwerp dient als blauwdruk voor de uiteindelijke realisatie van de module. De uiteindelijke interface kan visueel enigszins afwijken van deze concepten, omdat tijdens de daadwerkelijke realisatie voorrang verleend zal worden aan het hergebruik van bestaande UI-componenten en code-helpers binnen de applicatie. In figuur 1 wordt het conceptontwerp van het hoofddashboard weergegeven, inclusief de multi-select filters en export-functionaliteit, zoals deze in de applicatie gerealiseerd zou moeten worden.

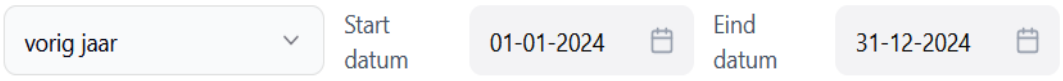


Figuur 1: Conceptontwerp van het dashboard met multi-select filters en export-functionaliteit.

3.1. Tijdsperiodeselectie

De rapportagedata zal strikt afhankelijk zijn van specifieke meetperiodes. Hiervoor zal een herbruikbare datepicker-component worden geïntegreerd die tevens in meerdere dashboards binnen het 'GBP' (Gebouw Beheer Portaal) wordt gebruikt. Zoals te zien in Figuur 2, krijgt de gebruiker hierbij twee opties:

- Presets: Snelle selectie van gestandaardiseerde blokken zoals 'Vorig jaar', 'Deze maand', 'Winterperiode' of 'Afgelopen 12 maanden'.
- Custom Range: Handmatige invoer van een 'Start datum' en 'Eind datum' via een kalender-interface voor specifieke audit-periodes.



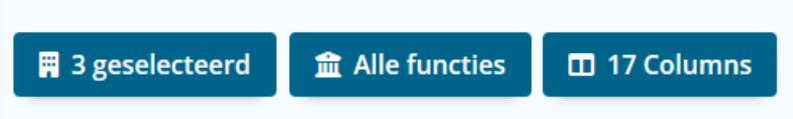
Figuur 2: Conceptontwerp van de tijdsperiodeselectie.

3.2. Dynamische filters & status

Om de gepresenteerde data overzichtelijk te houden, dient de interface gebruik te maken van dynamische filters. Deze functionaliteit is gevisualiseerd in Figuur 3 en Figuur 4 en omvat de volgende onderdelen:

- Locatie & functie: Via cascading dropdown-filters (zie Figuur 3) zou de weergave specifiek afgestemd moeten kunnen worden (bijv. uitsluitend kantoren in een specifieke regio). Zodra een filter is geactiveerd, wordt de selectiestatus visueel teruggekoppeld door het exacte aantal geselecteerde items in de knop te tonen.

- Reset-functie: De knop 'Nieuwe installatie' vervalt en zou vervangen moeten worden door een 'Wis alle filters' of 'Reset' knop (zie Figuur 4). Dit dient de tabel direct terug te zetten naar de volledige, ongefilterde dataset.



Figuur 3: Conceptontwerp van de actieve filterindicatoren.



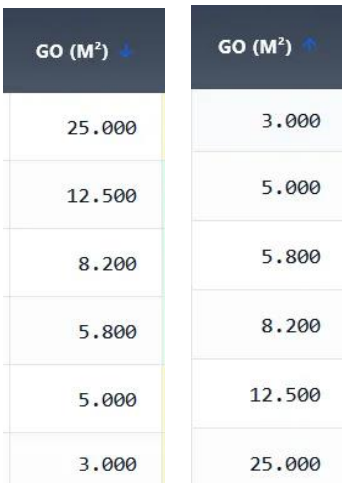
Figuur 4: Conceptontwerp van de reset-knop.

3.3. Tabelinteractie & sortering

Om de gebruikerservaring binnen het dashboard te optimaliseren, zijn er diverse interactiemogelijkheden in de tabel verwerkt.

Sorteer-logica: Elke kolomnaam zal worden voorzien van een interactief icoon (zie Figuur 5). Hiermee kan de gebruiker de data asynchroon oplopend of aflopend sorteren. Dit is cruciaal om bijvoorbeeld snel de gebouwen met het hoogste energieverbruik bovenaan te zetten.

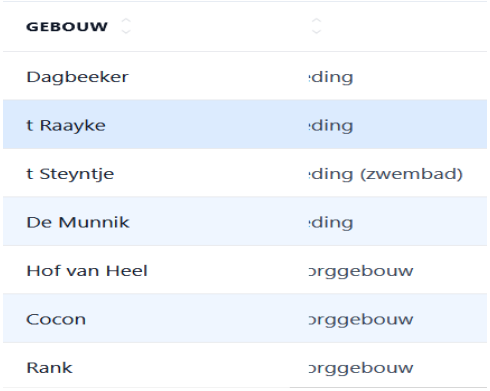
Kolomzichtbaarheid: Gezien de breedte van de dataset kan de gebruiker via een 'Kolommen'-menu handmatig datapunten aan- of uitzetten. Deze interactie wordt asynchroon afgehandeld zonder dat de pagina herladen hoeft te worden.



Figuur 5: Ontwerp van de interactieve sorteer-iconen in de tabelheaders.

3.4. Layout & scrolling

Vanwege de grote hoeveelheid kolommen krijgt de tabel standaard een horizontale scrollbar. Om de leesbaarheid (vooral op kleinere schermen) te garanderen, zal de kolom 'Gebouwnaam' worden vastgezet ofwel gepind (Figuur 6). Bij horizontaal scrollen blijven de namen van de gebouwen hierdoor altijd in beeld, terwijl de technische metrieken (zoals CO2, MJ, kWh) er soepel onderdoor schuiven.



Figuur 6: Ontwerp van de interactieve sorteer-iconen in de tabelheaders.

3.5. Data-integriteit & waarschuwingen

Een kernvereiste voor het ontwerp is het voorkomen van foutieve interpretaties op basis van de gepresenteerde data. In Figuur 7 is de visuele uitwerking hiervan te zien:

- Nul vs. Nil (Missing Data): Er wordt architecturaal een strikt onderscheid gemaakt tussen een legitieme nulmeting (bijv. daadwerkelijk 0 kWh opgewekte zonne-energie in de nacht) en het ontbreken van data (nil). Ontbrekende waarden worden nooit automatisch geconverteerd naar een '0', om te voorkomen dat gemiddelden scheef worden getrokken.

- Niet van toepassing (n.v.t.): Er wordt rekening gehouden met gebouwen waar bepaalde sensoren fysiek niet van toepassing zijn (denk aan de afwezigheid van gasverbruik bij een *all-electric* gebouw). In deze cellen wordt expliciet 'n.v.t.' getoond.

- Visual alert: Wanneer een meetwaarde die wel aanwezig hoort te zijn ontbreekt in de database, verschijnt er een waarschuwing met de melding 'Geen data'. Dit sluit interpretatiefouten uit.

- Kleurcodering via graph-logica: Om de leesbaarheid te vergroten en platformbrede consistentie te borgen, is het kleursysteem direct gekoppeld aan de bestaande graphs-logica. De drempelwaarden (lichtgroen, groen, oranje, rood) in de tabel komen hierdoor exact overeen met de gauges op de andere dashboards van de klant.



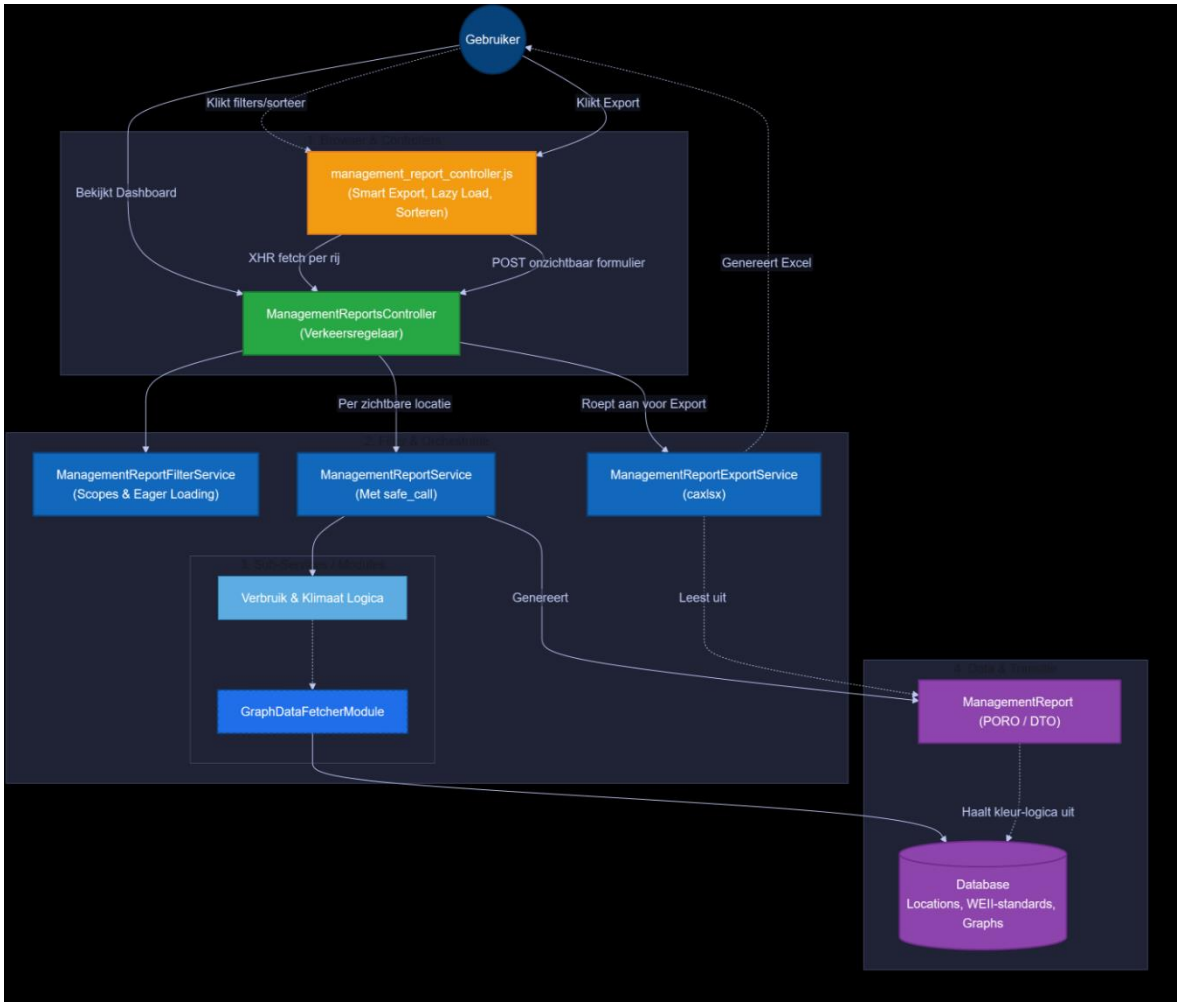
Figuur 7: Visuele weergave van de data-statussen, waarschuwingen en kleurcoderingen.

3.6. Export naar excel

De export-functie is c*ontext-aware*. De logica is zodanig ontworpen dat de exacte staat van de tabel in de view 1-op-1 wordt overgenomen in het Excel-bestand. Alleen de op dat moment zichtbare kolommen en geactiveerde filters worden geëxporteerd, inclusief de door de gebruiker gekozen sortering. Dit *What You See Is What You Get* principe voorkomt dat een gebruiker na de export handmatig aanpassingen hoeft te doen in Excel.

4. Technische realisatie

In dit hoofdstuk wordt de feitelijke bouw van de management rapportage module beschreven. De realisatie is uitgevoerd in verschillende iteratieve fases: van een initiële monolithische opzet naar een volledig ontkoppelde architectuur. Hierbij stonden drie pijlers centraal: schaalbaarheid, data-integriteit en een naadloze gebruikerservaring. Om de samenhang tussen deze verschillende onderdelen te visualiseren, is een architectuurschema opgesteld. In figuur 8 is het model te zien dat de datastroom van de applicatie weergeeft.



Figuur 8: C4-model weergave van de architectuur en datastromen binnen de Management Rapportage Tool.

4.1. Backend architectuur: Overzichtelijke en modulaire code

Een belangrijk onderdeel van de realisatie was de refactor van de ManagementReportsController. In de beginfase bevatte de controller veel "business logic", wat leidde tot een onoverzichtelijke structuur. Conform het *Single Responsibility Principle* is deze logica onttrokken naar gespecialiseerde service objects. De datastroom is nu als volgt georganiseerd:

1. ManagementReportFilterService: Verwerkt de inkomende parameters en vertaalt deze naar database-scopes.

2. ManagementReportService: De "orchestrator" die per gebouw de benodigde berekeningen (verbruik, klimaat, WEII) aanroept.

3. ManagementReport (PORO/DTO): In plaats van ruwe arrays, worden de resultaten verpakt in een *Plain Old Ruby Object*. Dit object fungeert als een Data Transfer Object (DTO) dat zowel door de HTML-view als de Excel-export op identieke wijze wordt uitgelezen.

De volledige architecturale vertaling en de resulterende mappen- en bestandsstructuur van deze service objects zijn uitgebreid gedocumenteerd in Bijlage A. In deze bijlage wordt de fysieke opbouw binnen de applicatie nader toegelicht, wat de modulaire structuur en de onderhoudbaarheid van de codebase concreet onderbouwt.

4.1.1. Scheiding van Verantwoordelijkheden

Om de applicatie schaalbaar en onderhoudbaar te houden, is de logica strikt opgesplitst in specifieke domeinen:

- Controllers: Er is bewust gekozen voor twee losse controllers. De ManagementReportsController beheert uitsluitend de visuele interactie (HTML weergave en Turbo Stream filters). De ManagementReportsExportController is afgesplitst en fungeert als een specifieke API-endpoint die uitsluitend JSON-payloads accepteert en Excel-bestanden teruggeeft. Dit voorkomt een zogeheten *fat controller*.

- Service objects: De zware bedrijfslogica is uit de controllers gehaald. De ManagementReportService functioneert als 'Orchestrator' die sub-services aanroept voor specifieke data (zoals klimaat of verbruik). Dit houdt de code modulair en testbaar.

- Helpers: Om de HTML-views schoon te houden, is weergavelogica verplaatst naar de ManagementReportsHelper. Deze helper is verantwoordelijk voor het formatteren van getallen (precisie) en het bepalen van de visuele status (kleuren en "n.v.t."-meldingen).

- Parameter casting & veiligheid aanvankelijk kwamen URL-parameters (zoals actieve filters en datums) binnen als onveilige, platte Strings. Binnen de nieuwe architectuur is er een ManagementReportFilterService gebouwd. Deze fungeert als een 'wasstraat' die de platte parameters direct bij binnenkomst parseert en cast naar veilige Ruby-objecten (zoals genormaliseerde arrays voor kolom-weergave en Time-objecten voor datums). Dit garandeert dat de achterliggende services altijd met schone data werken.

4.2. Snelheidsoptimalisatie en database-efficiëntie

Bij het inladen van rapportages voor grote vastgoedportfolio's ontstond het risico op een fundamenteel "N+1 query probleem" (AppSignal, 2018). Bij een initiële test probeerde elke rij in de tabel afzonderlijk gerelateerde data (zoals instellingen, mappen en datapunten) op te halen, wat de database zwaar belastte.

Om dit op te lossen is in eerste instantie *eager loading* toegepast in de controller:

```
  def base_location_scope
    policy_scope(Location).includes(
      :setting,
      :open_schedules,
      graphs: [:graph_sets, { folder: [:points, { folders: :points }] }]
    )
  end
```

Door deze optimalisatie worden alle benodigde relaties in slechts enkele queries opgehaald, ongeacht het aantal gebouwen in de rapportage.

Analyse van het fundamentele N+1 probleem
Tijdens de testfase bleek deze .includes structuur echter niet afdoende om alle N+1 queries op te lossen. Om de onderliggende oorzaak te begrijpen, is er in de Rails console een recursief script geschreven dat de daadwerkelijke data-architectuur van de folder_refs tabel uittekent. Dit is getest op de electricity_consumption_graph (Map ID: 2056):

```
def print_boom(folder, niveau = 0[CS10.1])
  puts ("  " * niveau) + "🗁 #{folder.name} (ID: #{folder.id})"
  folder.points.each { |point| puts ("  " * (niveau + 1)) + "🔑 Point:
#{point.name}" }
  folder.folders.each { |sub_map| print_boom(sub_map, niveau + 1) }
end

hoofdmap = Folder.find(2056)
print_boom(hoofdmap)
```

Resultaat (Console Output):

🗁 electricity_consumption_graph (ID: 2056)
  🗁 totals (ID: 2057)
    🗁 energy_used (ID: 2045)
      🔑 Point: verbruik_kwh/cumulative
    🗁 solar_energy (ID: 2047)
      🔑 Point: AnneFrank_cumulative_site_life_time_AnneFrank
    🗁 energy_returned (ID: 2059)
      🗁 energy_returned (ID: 2046)
        🔑 Point: teruglevering_kwh/cumulative

Conclusie & definitieve oplossing
Uit de console-output blijkt duidelijk dat de mappenstructuur binnen FourIQ een dynamische en variabele diepte heeft. Waar energy_used stopt op niveau 2, heeft energy_returned nog een extra submap op niveau 3 voordat het daadwerkelijke datapunt (Point) wordt bereikt. Omdat de diepte van de boomstructuur vooraf onbekend is, kan ActiveRecord deze niet met een statische .includes vooraf volledig inladen, wat de resterende database-queries verklaart.

4.2.1. Definitieve oplossingen

Om de stabiliteit en snelheid van de applicatie te garanderen, is er een architecturale afweging gemaakt tussen frontend- en backend-optimalisaties:

Frontend: asynchroon inladen
Om te voorkomen dat de server blokkeert tijdens een zware initiële laadactie, is er gekozen voor *Lazy Loading*. De applicatie laadt in eerste instantie slechts de basisstructuur van de tabel in. Vervolgens wordt elk gebouw asynchroon, via losse *Turbo Streams*, op de achtergrond berekend en ingeladen. Dit garandeert een razendsnelle *Time-to-First-Byte* voor de eindgebruiker, zelfs bij de maximale portfoliogrootte.

Backend: geheugenoptimalisatie
Uit log-analyses bleek dat bij het laden van elke individuele tabelrij onnodig zware data (zoals filteropties voor dropdown-menu's) in het servergeheugen werd geplaatst. Door de controller te refactoren en deze statische data uit te sluiten voor de asynchrone requests, is de belasting op het geheugen (*Garbage Collection*) met ruim 25% gedaald. Dit versnelde de verwerkingstijd per gebouw met gemiddeld 1,5 seconde.

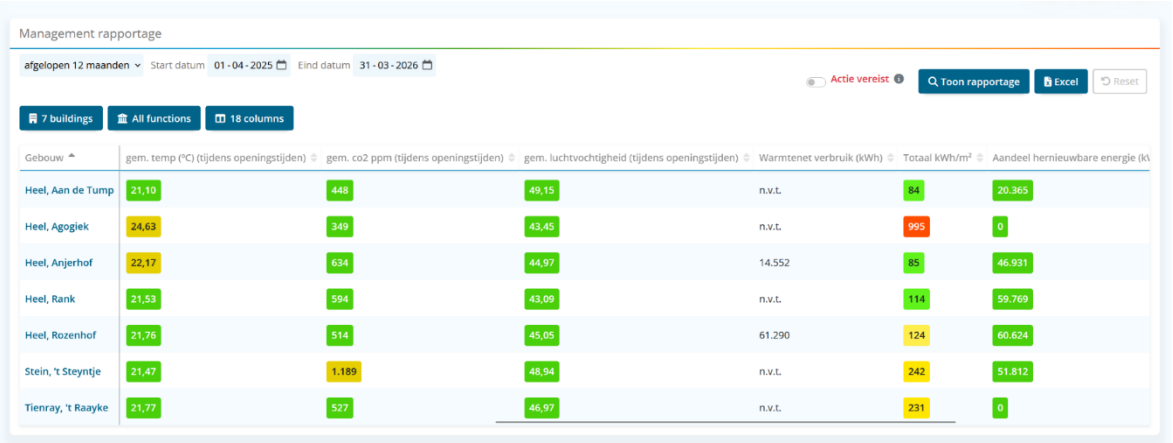
Architecturaal experiment: multithreading versus sequentiële verwerking
Tijdens de ontwikkelfase is geëxperimenteerd met het gelijktijdig ophalen van sensordata (*multithreading*) om de wachttijden verder te minimaliseren. Uit uitvoerige performancetests bleek echter dat de daadwerkelijke tijdswinst marginaal was. Dit fenomeen wordt verklaard door beperkingen binnen de programmeertaal (de *Global Interpreter Lock*), waardoor het zware rekenwerk alsnog niet volledig parallel kon worden uitgevoerd.

Bovendien introduceerden de actieve threads een kritiek architecturaal risico: bij het gelijktijdig inladen van tientallen gebouwen raakte de database overbelast, wat resulteerde in server-timeouts. Geconfronteerd met het feit dat de nadelen (risico op crashes) zwaarder wogen dan de minimale tijdwinst, is ervoor gekozen deze optimalisatie terug te draaien. De backend verwerkt de berekeningen nu bewust op een gecontroleerde, sequentiële manier.

4.3. Frontend realisatie

In deze paragraaf wordt de technische realisatie van de frontend behandeld. Het hoofddoel van de vernieuwde interface is om grote hoeveelheden gebouwdata efficiënt in te laden, zonder dat de browser blokkeert.

In Figuur 9 is de uiteindelijke weergave van dit dashboard te zien. Op deze afbeelding is goed zichtbaar hoe de tabelrijen stuk voor stuk (asynchroon) op de achtergrond worden geladen. Dit mechanisme zorgt ervoor dat de gebruiker direct met de interface kan interacteren, zelfs wanneer de achterliggende server nog bezig is met het berekenen van de overige datapunten.



Figuur 9: Gerealiseerde weergave van het dashboard waarbij tabelrijen asynchroon worden ingeladen.

Om de rapportagetool succesvol op te leveren, zijn diverse specifieke functionaliteiten en beveiligingen doorgevoerd. In de onderliggende paragrafen worden de belangrijkste technische keuzes achtereenvolgens behandeld, waaronder de foutafhandeling, veilige databerekeningen en het dynamische filtergedrag van de applicatie.

Uitbreiding functionaliteiten en transparantie Tijdens de verdere ontwikkeling van de frontend zijn er extra features gerealiseerd om de bruikbaarheid te vergroten. Zo zijn op verzoek van het management extra kolommen en KPI's uit bestaande dashboards succesvol geïntegreerd in de rapportage. Daarnaast is er een functionaliteit voor 'Filter Transparantie' ontwikkeld; hiermee is visueel inzichtelijk gemaakt welke specifieke gebouwen door de applicatie zijn weggefilterd (bijvoorbeeld wanneer de "Actie vereist"-toggle is geactiveerd).

4.3.1. View refactoring

Voortbouwend op de architecturale keuze voor componentisatie, is de view-structuur van het dashboard volledig herzien en opgesplitst in herbruikbare partials (zoals _filters.html.erb, _table.html.erb en _building_row.html.erb).

De _building_row.html.erb bevat uitsluitend de weergavelogica voor één enkele rij uit de tabel. Bij het laden van de hoofdpagina (index.html.erb) rendert Rails de tabel niet met data, maar plaatst het per gebouw een lege "container" (een <tbody> met een data-attribuut naar de juiste endpoint). Omdat de rij-logica netjes is geïsoleerd in een partial, kan de controller deze HTML-snippet geïsoleerd terugsturen wanneer de Stimulus-controller er via de Fetch

API om vraagt. Dit houdt de frontend-code overzichtelijk en de server-responses minimaal qua bestandsgrootte.

4.3.2. Frontend logica: De rapportage-controller

De Stimulus-controller beheert de levenscyclus van de dataweergave. De methode *loadBuildingRow* detecteert via de *tableBodyTargetConnected* callback wanneer een rij beschikbaar is in de DOM en haalt de data op via een asynchroon verzoek:

```
async loadBuildingRow(tbody) {
  const src = tbody.dataset.src
  if (!src || tbody.dataset.loaded === "true") return

  try {
    const response = await fetch(src, {
      headers: {
        "Accept": "text/html",
        "X-Requested-With": "XMLHttpRequest"
      }
    })
```

Dit mechanisme voorkomt een "request storm" op de server en zorgt ervoor dat de gebruiker direct interactie kan hebben met het dashboard terwijl de data per rij binnenstroomt.

4.3.3. UX-optimalisatie

Zoals in het functioneel ontwerp (hoofdstuk 2.8.3) is beschreven, vereist de interface dynamische *cascading filters*. Dit betekent dat de keuzes in de ene dropdown direct invloed hebben op de opties in de andere. Als een gebruiker bijvoorbeeld filtert op de functie 'Kantoor', mogen er in de locatie-dropdown geen zwembaden of sporthallen meer getoond (of geselecteerd) worden.
Binnen een traditionele applicatie zou elke wijziging in een filter een nieuwe server-request triggeren om de opties te updaten. Om de serverbelasting te minimaliseren en de gebruikerservaring razendsnel te houden, is deze logica volledig *client-side* gebouwd binnen de Stimulus-controller.

De technische werking: Alle gebouwen worden bij het laden van de pagina voorzien van een specifiek data-attribuut. De Stimulus-methode applyCascadingFilters() luistert continu naar wijzigingen in de functie-dropdown. Zodra de gebruiker een functie selecteert:

1. Controleert de JavaScript-logica welke gebouwen hieraan voldoen.
2. Worden niet-relevante gebouwen direct visueel verborgen in de DOM (display: none).
3. Data-integriteit: Belangrijker nog is dat de controller automatisch het vinkje (checkbox) weghaalt bij gebouwen die niet meer aan de criteria voldoen.

Deze grensvoorwaarden in de frontend voorkomen dat gebruikers per ongeluk conflicterende filters naar de backend sturen. Dit vereenvoudigt de interactie voor de gebruiker aanzienlijk en voorkomt complexe foutafhandeling aan de serverkant.

4.4. Stabiliteit

In dit hoofdstuk wordt beschreven hoe de applicatie robuust is gemaakt tegen foutieve of ontbrekende data. Tijdens de realisatie kwam een kritiek probleem aan het licht: gedeelde datamodellen binnen het platform crashten wanneer er rapportages werden gegenereerd voor nieuwbouwlocaties of gebouwen zonder actuele sensordata.

Tijdens de realisatie kwam een kritiek probleem aan het licht. Gedeelde datamodellen binnen het platform (zoals plot.rb in de fouriq_shared_models library) crashten met een *NoMethodError for nil:NilClass* wanneer er rapportages werden gegenereerd voor nieuwbouwlocaties of gebouwen zonder actuele sensordata.

Omdat een crash op één ontbrekende sensor de export van een volledig portfolio kon blokkeren, is in de eerste iteratie geprobeerd dit af te vangen met een *"catch-all"* rescue wrapper.

Na een interne code review is deze architectuur echter herzien. Het generiek "inslikken" van foutmeldingen (silent failures) is een anti-pattern dat debuggen in productie bemoeilijkt. De logica is daarom geherstructureerd met guard clauses in de upstream sub-services. Er wordt nu proactief gecontroleerd of de database ruwe samples bevat, vóórdat het berekeningsmodel wordt aangeroepen:

```
def call(graph_name, key)
  samples = fetch_samples_somehow(graph_name, key)

  # Failsafe: Voorkom dat plot.rb crasht op een lege dataset
  return nil if samples.blank?

  Plot.from_samples(samples: samples)
end
```

Door deze defensieve programmeerstijl isoleert een defecte of lege sensor zichzelf. Gecombineerd met geavanceerde "Safe Math" in de weergavelaag, resulteert dit in een robuuste interface: legitieme 0-waarden (bijv. 0 kWh zonne-opwek in de nacht) worden correct wiskundig verwerkt, terwijl écht ontbrekende data (nil) consequent als "n.v.t." of een visueel waarschuwingsicoon wordt getoond, zonder dat de batch-job crasht.

Data-consistentie en fallbacks in de praktijk
Tijdens het testen bij verschillende klanten (o.a. Daelzicht en Mosalira) bleek dat brondata niet altijd consistent is ingevuld. Om te voorkomen dat lege velden de lay-out verstoren of de tabel doen crashen, zijn robuustheidsverbeteringen doorgevoerd:

- Lege velden & Ruby .presence: In de building_row wordt gebruikgemaakt van de .presence methode. Als een veld (zoals plaatsnaam) leeg is, wordt dit veilig opgevangen met een I18n-vertaalde default waarde (bijv. "Plaats onbekend"). Hierdoor vallen er geen gaten in de sticky column.

- Gedeeltelijke data inladen: Indien een gebouw voor één specifiek meetpunt (bijv. gas) geen data heeft, wordt de rest van de rij alsnog volledig ingeladen. Alleen het ontbrekende punt toont een fallback (n.v.t.).

Ontkoppelen van UI-logica en configuratie De grenswaarden voor statuskleuren (goed, matig, slecht) zijn gelijkgetrokken met de rest van het platform. Door dynamisch de weii_standard uit de database-instellingen van een gebouw op te halen, kleuren de cellen in de rapportage nu op exact dezelfde wiskundige basis als de dashboards (gauges) elders in de applicatie. Dit verhoogt de uniformiteit.

4.5. Slimme Excel-export op basis van schermweergave

Een cruciale eis was dat de Excel-export exact de staat van het scherm moest overnemen. De technische uitdaging hierbij was de "state mismatch" tussen de client en de server. Omdat het filteren van gebouwen asynchroon in de browser gebeurt via CSS, is de server zich niet bewust van de actuele selectie van de gebruiker.

Om dit op te lossen is de exportToExcel functie in de Stimulus-controller volledig herschreven naar een "Smart Export". Deze functie voert de volgende stappen uit:

- State validatie & pre-filtering: De JavaScript controleert of de gebruiker handmatig gebouwen heeft aangevinkt. Zo niet, dan itereert het script over alle gebouw-rijen in de DOM en verzamelt het uitsluitend de ID's van de rijen die *zichtbaar* zijn (style.display !== "none"). Dit fungeert direct als een krachtige frontend-filter: gebouwen die door de gebruiker zijn weggefilterd (bijv. "Actie Vereist"), worden überhaupt niet meer naar de server gestuurd.

- JSON payload generatie: Waar in eerdere iteraties nog gebruik werd gemaakt van verborgen HTML-formulieren, is deze architectuur na een code review volledig gemoderniseerd. De Stimulus-controller verzamelt alle actieve parameters (datums, filter-ID's, actieve sortering) en bouwt hier een gestructureerd JSON-object van.

- Asynchrone blob download: De JSON-payload wordt via een POST-verzoek naar de ManagementReportExportService verstuurd. De Fetch API wacht asynchroon tot de server het Excel-bestand heeft gegenereerd. De binaire data (Blob) wordt vervolgens direct in het browsergeheugen opgevangen en omgezet naar een veilige download-URL, compleet met een dynamische bestandsnaam die uit de server-headers wordt geëxtraheerd.

Deze moderne implementatie elimineert kwetsbare DOM-manipulaties en cookie-polling, garandeert dat de backend nooit onnodige berekeningen uitvoert, en levert exact de dataset op die de gebruiker op zijn scherm ziet.

4.6. Toegangsbeveiliging en Gebruikersrechten

Een nieuwe datamodule brengt het risico met zich mee dat gebruikers toegang krijgen tot functionaliteiten of data waar zij niet voor geautoriseerd zijn, of waar zij niet voor betalen binnen hun licentie. Om dit af te dichten, is er een strikte ManagementReportPolicy geïmplementeerd (middels de Pundit gem Varvet, z.d.) die de toegang op drie niveaus reguleert als een absolute poortwachter:

1. Master Key: fouriq.nl accounts krijgen direct toegang voor intern beheer.
2. Tenant-niveau: De policy verifieert of de huidige klant (portal) op de 'allow-list' staat, wat de module 'plug-and-play' in- of uitschakelbaar maakt per klant.
3. Rol-niveau: Er wordt gecontroleerd of het actieve account de juiste rol heeft (bijv. admin of region_manager).

Architectuur: Om het DRY-principe (Don't Repeat Yourself) te respecteren, is de gedeelde autorisatielogica verplaatst naar het moederscript (ApplicationPolicy). Hierdoor kunnen toekomstige modules veilig van deze logica overerven, wat de schaalbaarheid en veiligheid van het platform vergroot.

5. Kwaliteitsborging en refactoring

n dit hoofdstuk wordt beschreven hoe de kwaliteit van de code en de gebruikerservaring gedurende de gehele realisatiefase is gewaarborgd. Kwaliteitsborging was binnen dit project een continu, iteratief proces. In de onderliggende paragrafen worden het code-reviewproces, de doorgevoerde UI/UX-verbeteringen, het geautomatiseerde testproces en een praktijkcasus over datakwaliteit behandeld

5.1. Code review proces & peer feedback

Tijdens de realisatiefase is de code continu onderworpen aan peer-reviews (via Pull Requests). Feedback van Senior Developers is direct doorgevoerd ter verhoging van de codekwaliteit en stabiliteit:

- Naamgevingsconventies: Om de context binnen de applicatie te borgen, is de volledige feature hernoemd (van BuildingReport naar ManagementReport). Ook zijn klasses domeinspecifiek gemaakt (bijv. van SimpleConsumptionService naar ManagementReportConsumptionService).

- Magic numbers: Directe rekenwaarden in de logica zijn geëxtraheerd naar constanten (zoals GAS_TO_KWH_FACTOR = 9.769) om de onderhoudbaarheid te vergroten en *Clean Code* principes (Martin, 2009) na te leven.

- Moderne data-overdracht (Van HTML Forms naar JSON): Tijdens de ontwikkeling van de 'Smart Export' werd initieel gebruikgemaakt van onzichtbare HTML-formulieren (<input type="hidden">) en cookie-polling om de export status te volgen. Een Senior Developer merkte in de Pull Request op dat het veel strakker en beter is om deze complexe parameter-lijsten als JSON-payload te versturen. De code is hierop succesvol gerefactord naar een moderne Fetch API-integratie, wat resulteerde in een aanzienlijk schonere en kortere codebase in de frontend.

- Performance en de "Catch-22" van database filtering: In de export-logica wees een reviewer op een N+1 / performance risico: de applicatie voerde zware berekeningen uit voor álle gebouwen, om ze vervolgens in Ruby pas te filteren op 'Actie Vereist'. De voorgestelde oplossing vanuit de peer-review was om dit direct in de database-query (ActiveRecord) af te vangen. Dit leidde echter tot een architecturale "Catch-22": de database weet pas of een gebouw "Actie Vereist" is, nádat de complexe WEII-berekeningen in Ruby zijn voltooid. Ik heb beargumenteerd dat deze performance-winst wél behaald kon worden, maar dan in de *frontend*. Doordat StimulusJS uitsluitend de ID's van de zichtbare (en dus reeds als slecht beoordeelde) gebouwen in de JSON-payload stopt, berekent de backend nu exact wat nodig is. Dit toont aan hoe frontend en backend architectuur naadloos kunnen samenwerken om serverbelasting te minimaliseren.

5.2. UI/UX Feedback en Iteratie

Naast de technische backend-reviews is er een specifieke frontend- en UI/UX-review uitgevoerd met een senior engineer. In plaats van de bevindingen op een langetermijn-backlog te plaatsen, is besloten deze direct in de huidige iteratie te integreren. De focus verschoof hierbij van een "werkend prototype" naar een "productiewaardige interface" die voldoet aan de hoge front-end-standaarden van FourIQ. De belangrijkste doorgevoerde optimalisaties zijn als volgt te categoriseren:

CSS refactoring & stabiliteit

- Van grid naar flexbox: De initiële CSS Grid-layout van de filtersectie is herschreven naar een stabielere Flexbox-oplossing (flex-wrap). Dit loste uitlijningsproblemen op en zorgt voor een consistenter responsive gedrag over verschillende browsers en apparaten.

- Code quality: Tijdens de review werd vastgesteld dat het gebruik van !important tags in de CSS vermeden moest worden. Dit is gerefactord door de CSS-specificiteit van de elementen (zoals hover-statussen op tabelrijen) te verhogen, wat resulteert in een schonere en beter onderhoudbare stylesheet.

Datavisualisatie & Tabel UI

- Uniforme 'card' layout: Om de module naadloos te integreren in de rest van de applicatie, is de rapportage in een standaard 'Card'-component geplaatst met een begrensde container. Hierdoor scrolt alleen de datatabel, terwijl de filters en paginering altijd vast in beeld blijven.

- Tabel-styling: De leesbaarheid is sterk verbeterd door de implementatie van *sticky columns* (het vastzetten van de gebouwnaam bij horizontaal scrollen) en het doortrekken van zebra-striping (even/oneven rijen) over de volledige breedte van de tabel. Tevens wordt data nu duidelijker gevisualiseerd met cel-achtergrondkleuren in plaats van gekleurde tekst.

Robuustheid & gebruiksgemak (UX)

- Intelligente default statussen: Bij het openen van de module staat de datepicker nu standaard op de 'afgelopen 12 maanden' en worden alle gebouwen getoond. De gebruiker ziet hierdoor direct waarde zonder eerst te hoeven configureren.

- Geavanceerde foutafhandeling: Systeemmeldingen (zoals fouten bij het asynchroon ophalen van data) worden nu netjes afgehandeld via gestileerde *toasts* in plaats van generieke browser alert() pop-ups.

- Contextuele hulp & meertaligheid (I18n): Complexe filters zijn voorzien van interactieve tooltips. Daarnaast zijn alle interface-teksten en tooltips direct opgenomen in het meertaligheidssysteem (I18n via nl.yml en en.yml), passend bij de brede doelgroep van het platform.

5.3. Geautomatiseerde unit-test

Binnen het portaal van FourIQ is RSpec de standaard voor testautomatisering. Om de stabiliteit van de complexe datatransformaties te garanderen, is deze techniek ook voor de nieuwe module toegepast (Mata, 2021).
Om de stabiliteit van de complexe datatransformaties te garanderen, zijn er een aantal unit tests geschreven voor de kerncomponenten. De testdekking focust zich met name op de ManagementReport (DTO), de ManagementReportBuilder en de ManagementReportExportService. Hierbij is geverifieerd of wiskundige randgevallen (zoals delen door nul bij oppervlaktes of het correct renderen van nil waarden) juist worden afgehandeld zonder de applicatie te laten crashen.

Testbestand	**Wat wordt getest**
management_report_spec.rb	Verificatie van nil-bewaring in de DTO, logica rondom WHOLE_NUMBER_COLUMNS en kleur/status bepalingen.
management_report_builder_spec.rb	Validatie van de format_val helper: correcte afhandeling van nil, preventie van division-by-zero en juiste afronding.
management_report_export_service_spec.rb	Controle of nil resulteert in een lege cel in Excel en verificatie van de juiste kolomvolgorde.

5.4. Controle datakwaliteit

Tijdens de test- en validatiefase van de nieuwe rapportagetool is de gegenereerde output vergeleken met historische klantrapportages om de rekenlogica te valideren. Hierbij kwam een specifieke data-anomalie aan het licht bij de locatie Anjerhof.
Constatering van het probleem: De geaggregeerde verbruikscijfers die door de nieuwe service objecten werden berekend, kwamen voor vrijwel alle locaties exact overeen met de historische data. Bij Anjerhof bleek echter specifiek de waarde voor Warmtenet verbruik (kWh) structureel af te wijken.

Onderzoek & context: Uit onderzoek van de infrastructuur bleek het volgende:

1. Anjerhof is onderdeel van een zogeheten HVK (Hoofdverdelerkast). Deze HVK fungeert als centrale verdeelkast die de stroom/warmte onderverdeelt over meerdere aanpalende 'hofjes'.
2. Bij het genereren van de rapportages kwamen de berekende waardes voor alle andere hofjes die op deze HVK zijn aangesloten, wél 100% overeen met het verwachte Warmtenet verbruik.
3. Uitsluitend de mapping of de onderliggende datastroom van de meter bij *Anjerhof* volgt een afwijkend patroon ten opzichte van de standaard verdeelsleutel.

Conclusie & Scope-afbakening: Aangezien de rekenmodule (ManagementReportService) voor de overige hofjes op dezelfde HVK correcte waardes retourneert, is geconcludeerd dat de rekenlogica van de gebouwde applicatie solide is. De afwijking bij Anjerhof is geen software-bug in de rapportage-module, maar een anomalie in de brondata, de fysieke meter-mapping of de klant-specifieke verdeelsleutel in de database.

6. Conclusie en reflectie

Met de oplevering van de Management Rapportage module is een complex proces geautomatiseerd dat voorheen veel handmatig en foutgevoelig werk opleverde. De applicatie is succesvol geëvolueerd van een initieel, monolithisch prototype naar een robuuste, ontkoppelde architectuur op basis van het *Service Object Pattern*.

Door de implementatie van *StimulusJS* en de *Fetch API* binnen het moderne Rails-ecosysteem (Hotwire), is aangetoond dat het mogelijk is om razendsnelle, asynchrone interfaces te bouwen zonder afhankelijk te zijn van zware JavaScript-frameworks. Daarnaast is door de introductie van een *Data Transfer Object* (DTO) en strikte *Guard Clauses* de data-integriteit sterk verbeterd, waardoor de applicatie veilig omgaat met ontbrekende of afwijkende meetwaarden.

Voor de organisatie van FourIQ en haar eindgebruikers betekent dit een directe optimalisatie van de bedrijfsvoering. Gebouwbeheerders hebben nu de touwtjes zelf in handen en kunnen zelfstandig, performant en foutloos dynamische rapportages genereren en exporteren. Tegelijkertijd ligt er onder de motorkap nu een schone, testbare codebase waar het interne development-team in de toekomst efficiënt op kan voortbouwen.

LITERATUURLIJST

AppSignal. (2018, 24 april). *ActiveRecord performance: the N+1 queries antipattern.* https://blog.appsignal.com/2018/04/24/active-record-performance-the-n-1-queries-antipattern.html

Brown, S. (2018). *The C4 model for visualising software architecture.* https://c4model.com

Hansson, D. H. (z.d.). *Ruby on Rails Doctrine.* https://rubyonrails.org/doctrine

Hotwire. (z.d.). *Stimulus Handbook: Introduction & Origin.* https://stimulus.hotwired.dev/handbook/introduction

i18next. (z.d.). *Testing i18next.* https://react.i18next.com/misc/testing

Martin, R. C. (2009). *Clean Code: A Handbook of Agile Software Craftsmanship.* Prentice Hall.

Mata, M. (2021, 3 mei). *Ruby on Rails testing with RSpec: writing your first tests.* Medium. https://medium.com/nerd-for-tech/ruby-on-rails-testing-with-rspec-writing-your-first-tests-6330920928fd

Mix & Go. (z.d.). *Factory Method pattern in Ruby.* https://mixandgo.com/learn/ruby/factory-method-pattern

Norman, J. (2023). *Service Objects in Ruby on Rails.* Jardo.dev. https://jardo.dev/rails-service-objects

Refactoring.Guru. (z.d.). *Factory Method in Ruby: example.* https://refactoring.guru/design-patterns/factory-method/ruby/example

Ruby on Rails Discussions. (2022). *Revert turbo-frame to loading/pending state.* https://discuss.rubyonrails.org/t/revert-turbo-frame-to-loading-pending-state/82623

Stack Overflow. (2009). *NS_BINDING_ABORTED shown in Firefox with HttpFox.* https://stackoverflow.com/questions/704561/ns-binding-aborted-shown-in-firefox-with-httpfox

Thomas, D., & Hunt, A. (1999). *The Pragmatic Programmer: From Journeyman to Master.* Addison-Wesley.

Varvet. (z.d.). *Pundit: Minimal authorization through OO design and pure Ruby classes.* GitHub. https://github.com/varvet/pundit

BIJLAGE A — BESTANDSSTRUCTUUR MANAGEMENT RAPPORTAGE MODULE.

```
app/
├── assets/javascript/fouriq_portals_engine/controllers/
│   └── filter_list_controller.js
│   ├── sortable_table_controller.js
│   └── excel_export_controller.js
├── controllers/
│   ├── management_reports_controller.rb
│   └── management_reports_export_controller.rb
├── helpers/fouriq_portals_engine/
│   └── management_reports_helper.rb
├── models/
│   └── management_report.rb
├── modules/
│   └── graph_data_fetcher_module.rb
├── policies/
│   └── management_report_policy.rb
├── services/
│   ├── cost_graph_data_service.rb
│   ├── electricity_data_service.rb
│   ├── management_report_builder.rb
│   ├── management_report_climate_service.rb
│   ├── management_report_consumption_service.rb
│   ├── management_report_export_service.rb
│   ├── management_report_filter_service.rb
│   ├── management_report_search_service.rb
│   ├── management_report_service.rb
│   ├── peak_data_service.rb
│   └── solar_data_service.rb
└── views/management_reports/
    ├── _building_row.html.erb
    ├── _dropdown_filter.html.erb
    ├── _filtered_results.html.erb
    ├── _filters.html.erb
    ├── _table.html.erb
    ├── filter.turbo_stream.erb
    └── index.html.erb

config/locales/
├── en.yml
└── nl.yml

spec/
├── models/
│   └── management_report_spec.rb
└── services/
    ├── management_report_builder_spec.rb
    └── management_report_export_service_spec.rb
```